Voyage-AI — Testing Strategy & Software Quality Assessment

Document Reference: `docs/reports/testing-and-quality.pdf` / `docs/reports/testing-and-quality.md`
System Version: 0.2.1 (Phase 2 Verified Baseline)
Author: Senior QA Engineer & Quality Architect
Audit Date: October 25, 2026
Status: Implementation-Verified

---

1. Testing Frameworks, Tools & Quality Architecture

The Voyage-AI verification strategy employs a multi-tiered, deterministic testing pyramid designed to ensure rapid feedback, complete isolation of persistent stores, and zero reliance on fragile external networks during routine automated testing:

```
+-------------------------------------------------------------------------------------+
|                                  TESTING TOOLING MATRIX                             |
+-------------------------------------------------------------------------------------+
| Testing Layer              | Technology / Tool        | Version    | Primary Scope      |
|----------------------------|--------------------------|------------|--------------------|
| Backend Unit & Controller  | JUnit 5 / Spring MockMvc | 5.10+ / 4.1| HTTP mapping & err |
| Backend Mocking            | Mockito / MockitoBean    | 5.x        | Provider boundary  |
| Backend Assertions         | AssertJ                  | 3.25+      | Fluent assertions  |
| Database Integration       | Flyway / H2 In-Memory    | 10.x / 2.4 | Isolated schema V0-4|
| Frontend Component Unit    | Vitest / jsdom           | 2.1.9 / 25 | React DOM rendering|
| Frontend DOM Assertions    | React Testing Library    | 16.0.1     | User-centric events|
| Browser E2E Automation     | Java Playwright          | 1.48.0     | Multi-viewport UI  |
| Black-Box API Verification | Java HttpClient + AssertJ| Java 17 LTS| HTTP 200/400/403   |
+-------------------------------------------------------------------------------------+
```

---

2. Test Directory Structure & Organization

```
Voyage-AI/
├── backend/src/test/java/com/voyageai/
│   ├── VoyageAiApplicationTests.java                 [Application Context Smoke Test (1 test)]
│   ├── auth/
│   │   ├── AuthApiTest.java                          [MockMvc: Register, Login, Duplicate 409, Me, JWT (10 tests)]
│   │   └── UserApiTest.java                          [MockMvc: Profile Sync & Retrieval (1 test)]
│   ├── destination/
│   │   ├── DestinationApiTest.java                   [MockMvc: Search, Region/Budget Filters, Listings (9 tests)]
│   │   ├── AdminDestinationControllerTest.java       [MockMvc: Admin Auth 401, 403 User, 200 Admin (3 tests)]
│   │   └── DestinationIngestionServiceTest.java      [Unit: Wikipedia & Wikimedia Licensing Ingestion (13 tests)]
│   ├── trip/
│   │   └── TripApiTest.java                          [MockMvc: Trip CRUD, Days/Items, Ownership Isolation (8 tests
│   ├── integration/
│   │   ├── ai/
│   │   │   ├── AiControllerTest.java                 [MockMvc: /api/v1/ai/generate, /health, Error Codes (6 tests)
│   │   │   └── GeminiServiceTest.java                [Unit: Prompt Framing, Timeouts, Rate Limits (8 tests)]
│   │   ├── db/
│   │   │   ├── DatabaseFlywayIntegrationTest.java    [Flyway H2 Isolation & Schema Safety V0-V4 (4 tests)]
│   │   │   └── PersistenceIntegrationTest.java       [JPA: Foreign Keys, Cascades, Timestamps (3 tests)]
│   │   └── weather/
│   │       ├── OpenMeteoWeatherServiceTest.java      [MockRestServiceServer: Serialization, Caching (6 tests)]
│   │       └── WeatherControllerTest.java            [MockMvc: Coordinates, Parameter & Error Codes (7 tests)]
│   └── common/health/
│       └── HealthControllerTest.java                 [MockMvc: /api/v1/health & Actuator (4 tests)]
│
├── frontend/src/
│   ├── setupTests.ts                                 [@testing-library/jest-dom Matchers]
│   ├── App.test.tsx                                  [Vitest: Layout, Filters, Modals, Pagination (4 tests)]
│   └── components/
│       ├── common/ImageWithFallback.test.tsx         [Vitest: Image Load & SVG Fallback (3 tests)]
│       └── ai/AiTravelAssistant.test.tsx             [Vitest: Prompt Input, Generation Trigger (4 tests)]
│
└── tests/                                            [Centralized Automated Test Suite]
    ├── pom.xml                                       [Maven Java Playwright Configuration]
    ├── README.md                                     [Test Suite Documentation & Guides]
    └── src/test/java/com/voyageai/tests/
        ├── api/
        │   ├── AiApiTest.java                        [Direct AI API: Health, Validation, Live Gemini (3 tests)]
        │   ├── DestinationIngestionApiTest.java      [Wikimedia Ingestion API & Role Enforcement (1 test)]
        │   ├── HealthApiTest.java                    [Black-Box HTTP Health Checks (2 tests)]
        │   ├── PersistenceTripApiTest.java           [Trip Lifecycle, Days, 403 Cross-User, 401 Unauth (3 tests)]
        │   ├── UserProfileApiTest.java               [User Profile Sync, Preferences, Role Security (2 tests)]
        │   └── WeatherApiTest.java                   [Black-Box HTTP Open-Meteo Checks (3 tests)]
        ├── base/
        │   └── BasePlaywrightTest.java               [Playwright Lifecycle & Tracing]
        ├── browser/
        │   ├── AiAssistantBrowserTest.java           [Playwright: Live Gemini Round-Trip, New Delhi Verified (3 te
        │   ├── DestinationDetailModalTest.java       [Modal Tabs, Media Credit Links, Accessibility (4 tests)]
        │   ├── DestinationsDiscoveryTest.java        [Card Grid, Server Search, Region Filters, Pagination (5 test
        │   ├── NavigationTest.java                   [Header, Tabs, Health Dot, Auth Trigger (3 tests)]
        │   └── ResponsiveTest.java                   [Desktop 1280x800 vs Mobile 390x844 Layouts (2 tests)]
        └── e2e/
            └── WeatherE2ETest.java                   [Route Interception & Live Forecasts (3 tests)]
```

3. Verified Test Execution Results

The following test suites were executed and verified directly on the workstation host during the verification session on **October 25, 2026**:

3.1 Suite Summary Breakdown

```
================================================================================================
Test Module               Test Scope                 Total   Passed  Failed   Skipped
================================================================================================
backend/                  Unit, Domain & Controllers    83       83       0         0
frontend/                 Vitest Component Units        11       11       0         0
tests/                    Playwright Browser & APIs     34       34       0         0
------------------------------------------------------------------------------------------------
TOTAL VOYAGE-AI SUITE                                  128      128       0         0
================================================================================================
Pass Rate: 100.0%
```

3.2 Backend Test Execution Audit (`mvn test` in `backend/`)

- **Total Tests Run:** 83
- **Passed:** 83 | **Failed:** 0 | **Errors:** 0 | **Skipped:** 0 | **Status: BUILD SUCCESS**
- **Test Class Inventory:**
- `AdminDestinationControllerTest` : 3 passed (401 unauthenticated, 403 regular user `ROLE_USER` , 200 admin `ROLE_ADMIN`)
- `AuthApiTest` : 10 passed (Registration, login, duplicate email 409, profile retrieval, JWT validation)
- `DestinationApiTest` : 9 passed (Full-catalog search, filtering, category listings, pagination, provenance)
- `DestinationIngestionServiceTest` : 13 passed (Wikipedia summary extraction, MediaWiki Action API imageinfo CC licensing, duplicate rejection, coordinate validation)
- `TripApiTest` : 8 passed (Trip CRUD, day creation, ownership boundaries, budget summaries)
- `AiControllerTest` : 6 passed (Controller endpoints, input validation, error code translation)
- `GeminiServiceTest` : 8 passed (Prompt formatting, timeout handling, rate limit handling)
- `OpenMeteoWeatherServiceTest` : 6 passed (Weather parsing, cache hits, zero fabrication fallback)
- `WeatherControllerTest` : 7 passed (Coordinates parameters, missing coordinates 400, service 503)
- `HealthControllerTest` : 4 passed (App health, info endpoints)
- `DatabaseFlywayIntegrationTest` : 4 passed (Migrations V0-V4, table schemas, seed integrity, DB safety guard)
- `PersistenceIntegrationTest` : 3 passed (Cascade relationships, foreign key constraints, timestamps)
- `VoyageAiApplicationTests` : 1 passed (Spring context bootstrap)
- `UserApiTest` : 1 passed (Profile retrieval and sync)

3.3 Frontend Test Execution Audit (`npm run test:unit` in `frontend/`)

- **Total Tests Run:** 11
- **Passed:** 11 | **Failed:** 0 | **Skipped:** 0 | **Status: PASS**
- **Test Class Inventory:**
- `App.test.tsx` : 4 passed (Navigation, hero section, filter bar interaction, modal triggers)
- `ImageWithFallback.test.tsx` : 3 passed (Image rendering, broken image SVG plate fallback, accessibility)
- `AiTravelAssistant.test.tsx` : 4 passed (Initial empty state, prompt input validation, submit trigger, health badge)

3.4 End-to-End Browser & Black-Box API Audit (`mvn test "-Dtest.excludedGroups="` in `tests/`)

- **Total Tests Run:** 34

- **Passed:** 34 | **Failed:** 0 | **Skipped:** 0 | **Status: BUILD SUCCESS**
- **Test Highlights:**
- **Live Google Gemini Verification:** `AiAssistantBrowserTest.testRealGeminiApiGenerationViaBrowser` submitted prompt `"What's capital of India?"`, verified backend call to model `gemini-3.5-flash-lite`, and confirmed the response `"The capital of India is **New Delhi**."` rendered in Chromium DOM.
- **Live Open-Meteo Weather Verification:** `WeatherApiTest` verified live coordinate queries for New Delhi (`28.6139, 77.2090`), receiving valid temperature and conditions under CC-BY 4.0 license.
- **Admin Security Verification:** `DestinationIngestionApiTest` verified that standard users receive `403 Forbidden` and administrators receive `200 OK`.
- **Multi-Tenant Ownership:** `PersistenceTripApiTest` confirmed cross-user trip access returns `403 Forbidden`.
- **Catalog Pagination:** `DestinationsDiscoveryTest` verified server pagination controls and `X-Total-Count` headers.

---

4. Test Isolation & Database Safety Guarantees

To ensure automated tests cannot inadvertently corrupt developer databases (`voyage`) or host databases (`pat`), `DatabaseFlywayIntegrationTest.java` implements programmatic assertions:

```
// backend/src/test/java/com/voyageai/integration/db/DatabaseFlywayIntegrationTest.java:68-77
assertThat(url)
    .as("Safety check: Database URL must be an isolated in-memory database")
    .contains("jdbc:h2:mem:voyage_flyway_test");

assertThat(url.toLowerCase())
    .as("Safety violation prevention: Must NEVER point to host voyage or pat database")
    .doesNotContain("localhost:3306/voyage")
    .doesNotContain("localhost:3306/pat")
    .doesNotContain(":3307/voyage")
    .doesNotContain(":3307/pat");
```

Result: Verified active and passing. Zero host database connections or destructive alterations occur during test execution.

---

5. Local Test Execution & Development Orchestrator

Developers can launch the complete local runtime environment (MySQL, Spring Boot backend, and Vite frontend) using the single-command orchestrator:

```
start.bat
```

This script automatically executes dependency checks, polls health endpoints until UP, and opens the frontend in the default browser.